Progetto Rust: Ruggine
Descrizione Tecnica e Manuale Utente
Gruppo 44

https://github.com/PdS2425-C2/G44
Luca Lodesani: 346978
Alberto Nuzzaci: 346977
Riccardo Vaccari: 348856
Alessio Perrotti: 356737

1 - Architettura Generale

Il progetto Ruggine consiste nello sviluppo di un'applicazione client/server per la gestione di una chat testuale in tempo reale, che consente a più utenti di comunicare tra loro sia tramite conversazioni private sia tramite gruppi.

L'architettura del sistema segue il modello client-server

- **Client** → realizzato con la libreria React si occupa della presentazione dei dati e dell'interazione con l'utente, offrendo un'interfaccia intuitiva per la gestione delle chat, dei messaggi e degli inviti.
- **Server** → implementato in Rust, rappresenta il cuore logico dell'applicazione. Gestisce l'autenticazione degli utenti, l'elaborazione delle richieste, la persistenza dei dati e la comunicazione in tempo reale.

La comunicazione tra client e server avviene attraverso due meccanismi complementari:

- **API REST** → utilizzate per operazioni standard come autenticazione, recupero dati e gestione delle risorse;
- **WebSocket** → impiegati per la propagazione in tempo reale degli eventi (es. nuovi messaggi o notifiche).

Per la persistenza è stato utilizzato un database SQLite scelto per la sua leggerezza e semplicità di integrazione.

2 - Descrizione Tecnica

2.1 - Server

Il server di **Ruggine** è stato sviluppato in Rust come applicazione backend responsabile della gestione dell'autenticazione, della logica applicativa, della persistenza dei dati e delle notifiche in tempo reale. L'architettura adottata è modulare: ogni gruppo di funzionalità è isolato in file dedicati, così da migliorare leggibilità, manutenibilità e separazione delle responsabilità.

2.1.1 - Librerie Utilizzate

Il backend si basa su un insieme di librerie che coprono routing web, accesso al database, autenticazione, gestione dei cookie e comunicazione asincrona.

- **`Axum`** → è il framework web utilizzato per definire gli endpoint REST e l'endpoint WebSocket. Permette di costruire il router del server e associare ogni route al relativo handler asincrono. Nel progetto viene impiegato per gestire sessioni, utenti, chat, messaggi, richieste e notifiche.

- **`Tokio`** → costituisce il runtime asincrono su cui si basa l'intero server. Viene usato sia per l'esecuzione del web server sia per la gestione concorrente delle connessioni WebSocket e delle primitive di sincronizzazione asincrone.
- **`SQLx`** → è la libreria adottata per l'accesso al database SQLite. Consente di eseguire query SQL asincrone, creare transazioni e mappare i risultati verso strutture Rust e DTO.
- **`tower-cookies`** → gestisce i cookie HTTP lato server. In particolare viene utilizzata per leggere, creare e rimuovere il cookie di sessione `sid`, usato per identificare l'utente autenticato nelle richieste protette.
- **`tower-http`** → Viene usata per aggiungere middleware HTTP, in particolare il `TraceLayer`, che permette di tracciare le richieste e le risposte del server a fini di logging e debugging.
- **`Serde`** / **`serde_json`** → sono impiegate per serializzare e deserializzare i dati scambiati tra client e server in formato JSON. I DTO definiti in `dto.rs` derivano `Serialize` e `Deserialize`, mentre `serde_json` viene usata anche per costruire le notifiche WebSocket.
- **`Futures`** → la libreria `futures` è utilizzata nel modulo WebSocket per gestire stream e sink asincroni.
- **`Argon2`** / **`password-hash`** / **`Rand`** → queste librerie sono usate per la verifica sicura delle password durante il login e per la generazione degli hash nel tool di supporto `mkhash.rs`.
- **`Tracing`** / **`tracing-subscriber`** → queste librerie gestiscono il sistema di logging del server.
- **`Dotenvy`** → Viene usata per caricare da file `.env` le variabili d'ambiente necessarie all'avvio del server, come il percorso del database, il secret per i cookie e l'indirizzo di binding.

2.1.1 - Organizzazione dei file nel backend

Il file **`mod.rs`** funge da punto di aggregazione, esponendo i principali moduli del package **`routes/`**.

- **`main.rs`** → rappresenta il punto di ingresso dell'applicazione. Si occupa dell'inizializzazione del logging, del caricamento della configurazione, della creazione della pool di connessioni al database e dello stato globale condiviso. Inoltre definisce e registra tutte le route REST e WebSocket, avviando il server in ascolto.
- **`dto.rs`** → contiene le strutture dati (DTO) utilizzate per lo scambio di informazioni tra client e server. Include sia le strutture di output (utenti, chat, messaggi, richieste) sia i payload di input per le API.
- **`sessions.rs`** → gestisce il sistema di autenticazione e sessione. Implementa le operazioni di login, logout e verifica dell'utente autenticato, utilizzando cookie firmati e hashing delle password.
- **`users.rs`** → fornisce le funzionalità di ricerca degli utenti, consentendo di individuare altri utenti nel sistema per l'avvio di chat o inviti.

- **`chats.rs`** → gestisce tutta la logica relativa alle conversazioni, tra cui creazione di chat di gruppo e private, recupero delle chat dell'utente, gestione dei partecipanti, aggiornamento dello stato di lettura e uscita dai gruppi.
- **`messages.rs`** → si occupa dell'invio e del recupero dei messaggi. Verifica l'appartenenza dell'utente alla chat e, dopo l'inserimento nel database, propaga i messaggi agli altri partecipanti tramite WebSocket.
- **`requests.rs`** → implementa il sistema di inviti ai gruppi, permettendo di inviare richieste, visualizzarle e gestirne l'accettazione o il rifiuto.
- **`ws.rs`** → gestisce le connessioni WebSocket per le notifiche in tempo reale. Mantiene una struttura condivisa delle connessioni attive e inoltra gli eventi ai client collegati.
- **`mod.rs`** → dichiara ed esporta i moduli del backend, consentendo un'organizzazione centralizzata del codice.
- **`mkhash.rs`** → programma ausiliario utilizzato per generare hash di password durante la fase di sviluppo.

2.2 - Database

Il database è strutturato per rappresentare le principali entità del sistema: utenti, chat, partecipazioni, messaggi e richieste di invito.

- Tabella **USER**: Contiene le informazioni degli utenti registrati
    - **id** → identificativo univoco dell'utente (chiave primaria, autoincrementale)
    - **name** → nome visualizzato dell'utente
    - **username** → identificativo univoco scelto dall'utente (vincolo UNIQUE)
    - **password_has** → hash della password, utilizzato per l'autenticazione sicura
- Tabella **CHAT**: Rappresenta le conversazioni
    - **id** → identificativo univoco della chat (chiave primaria)
    - **name** → nome della chat (utilizzato solo per i gruppi, nullo nelle chat private)
    - **created_at** → timestamp di creazione della chat
    - **is_group** → booleano che distingue gruppo (true) da chat private (false)
- Tabella **ASSOCIATION**: Modella la partecipazione degli utenti alle chat (relazione molti-a-molti)
    - **user_id** → riferimento all'utente partecipante (chiave esterna)
    - **chat_id** → riferimento alla chat (chiave esterna)
    - **join_at** → timestamp di ingresso dell'utente nella chat
    - **last_read_at** → timestamp dell'ultima lettura dei messaggi

  Chiave primaria composta: *(user_id, chat_id)*
- Tabella **MESSAGE:** Contiene i messaggi scambiati nelle chat:
    - **id** → identificativo univoco del messaggio (chiave primaria)
    - **content** → contenuto testuale del messaggio

    - **user_id** → identificativo dell'utente mittente (chiave esterna)
    - **chat_id** → identificativo della chat di appartenenza (chiave esterna)
    - **sent_at** → timestamp di invio del messaggio

- Tabella **REQUEST**: Gestisce gli inviti alle chat di gruppo

    - **id** → identificativo univoco della richiesta (chiave primaria)
    - **chat_id** → chat a cui si riferisce l'invito (chiave esterna)
    - **inviter_id** → utente che invia l'invito (chiave esterna)
    - **invitee_id** → utente che riceve l'invito (chiave esterna)
    - **created_a** → timestamp di creazione della richiesta

2.3 - Client

Il client di Ruggine è stato sviluppato con la libreria React e si occupa della presentazione dei dati e dell'interazione con l'utente. La comunicazione con il server avviene attraverso due canali: le API REST, utilizzate per le operazioni standard su dati statici, e i WebSocket nativi del browser, impiegati per la ricezione in tempo reale di messaggi e notifiche. Tutta la logica di comunicazione con il server è centralizzata nel file `client.js`, che espone un oggetto `API` con metodi asincroni basati sulla Fetch API nativa del browser.

2.3.1 - Librerie Utilizzate

- **React** → è la libreria principale utilizzata per costruire l'interfaccia utente. Permette di strutturare l'applicazione in componenti riutilizzabili e di gestire lo stato tramite hook come **`useState`**, **`useEffect`**, **`useContext`** e **`useCallback`**.
- **React Router DOM** → gestisce il routing lato client. Consente di definire le rotte dell'applicazione (pagina di login e home) e di proteggere l'accesso alle pagine in base allo stato di autenticazione dell'utente.
- **React Bootstrap / Bootstrap** → fornisce i componenti UI già stilizzati utilizzati nell'interfaccia (modal, form, navbar, spinner, ecc.), velocizzando lo sviluppo e garantendo un'esperienza visiva coerente.
- **Bootstrap Icons** → libreria di icone vettoriali integrata con Bootstrap, utilizzata per arricchire l'interfaccia con elementi visivi come icone per le chat, inviti e azioni dell'utente.
- **WebSocket nativo** → per la comunicazione in tempo reale non è stata adottata alcuna libreria di terze parti: vengono utilizzate direttamente le API WebSocket native di JavaScript, mantenendo le dipendenze al minimo.

2.3.2 - Organizzazione dei file nel front end

Il codice del client è organizzato in cartelle che separano le responsabilità, seguendo le convenzioni tipiche dei progetti React.

```
.
├── eslint.config.js
├── index.html
├── package.json
├── package-lock.json
├── public
│   └── vite.svg
├── README.md
├── src
│   ├── api
│   │   └── client.js
│   ├── app
│   │   ├── App.jsx
│   │   └── routes.jsx
│   ├── assets
│   │   └── react.svg
│   ├── components
│   │   ├── auth
│   │   │   └── LoginForm.jsx
│   │   ├── feedback
│   │   │   └── MessageAlert.jsx
│   │   ├── groups
│   │   │   ├── ChatRoom.jsx
│   │   │   ├── CreateGroupModal.jsx
│   │   │   ├── CreatePrivateChatModal.jsx
│   │   │   ├── GroupsList.jsx
│   │   │   └── LeaveGroupModal.jsx
│   │   ├── invitations
│   │   │   └── InvitationModal.jsx
│   │   ├── layout
│   │   │   ├── DefaultLayout.jsx
│   │   │   └── NavHeader.jsx
│   │   └── users
│   │       └── UserSelect.jsx
│   ├── hooks
│   │   ├── GroupsProvider.jsx
│   │   ├── NotificationsProvider.jsx
│   │   ├── useAuth.jsx
│   │   └── useGroups.jsx
│   ├── main.jsx
│   └── pages
│       ├── Home.jsx
│       └── LoginPage.jsx
└── vite.config.js
```

- **`main.jsx`** → è il punto di ingresso dell'applicazione. Si occupa del rendering del componente radice nel DOM, inizializzando i provider globali (`AuthProvider`, `NotificationsProvider`, `GroupsProvider`) e il router.
- **`App.jsx`** → componente radice che delega direttamente il rendering al modulo di routing.
- **`routes.jsx`** → definisce le rotte dell'applicazione tramite `AppRoutes`. Gestisce la navigazione protetta (redirect al login se non autenticato), il rendering del layout e la logica di accettazione/rifiuto degli inviti.
- **`api/client.js`** → centralizza tutte le chiamate HTTP al server tramite un wrapper attorno alla Fetch API. Gestisce uniformemente gli errori, le risposte vuote (204) e la serializzazione JSON.
- **`pages/`** → contiene le pagine principali: `Home.jsx`, che gestisce la visualizzazione delle chat e della chat room selezionata, e `LoginPage.jsx`, che ospita il form di autenticazione.
- **`hooks/`** → raccoglie la logica di stato condivisa tra i componenti sotto forma di custom hook e context provider: `useAuth.jsx` gestisce l'autenticazione, `GroupsProvider.jsx` e `useGroups.jsx` lo stato delle chat, `NotificationsProvider.jsx` gli inviti e i messaggi in arrivo via WebSocket.
- **`components/`** → contiene tutti i componenti React organizzati per dominio: `layout/` (struttura della pagina e navbar), `auth/` (form di login), `groups/` (lista chat, chat room, modali di creazione e uscita), `invitations/` (modale degli inviti), `users/` (ricerca utenti), `feedback/` (alert di sistema).

2.4 - Prestazioni

2.4.1 - Consumo CPU

Come richiesto dalle specifiche del progetto, il consumo di CPU del server viene monitorato periodicamente. In particolare, è stato implementato un task asincrono separato dal flusso principale dell'applicazione, il quale si occupa di rilevare l'utilizzo della CPU ogni 2 minuti.

Questo task viene avviato all'inizializzazione del server e opera in maniera concorrente rispetto alla gestione delle richieste dei client, senza interferire con il normale funzionamento dell'applicazione. Tale scelta è coerente con il modello asincrono adottato (basato su Tokio), permettendo di evitare blocchi del thread principale.

Per ottenere i dati di utilizzo della CPU è stata utilizzata la libreria **sysinfo**, che consente di accedere alle informazioni relative al processo in esecuzione. In particolare, viene monitorata la percentuale di

utilizzo della CPU del processo del server, che rappresenta una misura significativa del carico computazionale generato dall'applicazione.

I valori rilevati vengono salvati su file (*cpu_log.txt*) in modalità append, così da mantenere uno storico delle misurazioni nel tempo.

```
[2026-04-03 14:16:15.043906 UTC] CPU usage: 0.00%
[2026-04-03 14:18:15.045285 UTC] CPU usage: 0.00%
[2026-04-03 14:20:15.046637 UTC] CPU usage: 0.15%
[2026-04-03 14:22:15.048347 UTC] CPU usage: 0.31%
[2026-04-03 14:24:15.049561 UTC] CPU usage: 0.09%
[2026-04-03 14:26:15.050985 UTC] CPU usage: 0.47%
[2026-04-03 14:28:15.054304 UTC] CPU usage: 0.05%
```

Poiché il sistema è stato testato in un ambiente non soggetto a carichi elevati o traffico reale, i valori di utilizzo della CPU risultano generalmente contenuti. Tuttavia, la soluzione adottata è scalabile e consente di osservare il comportamento del sistema anche in condizioni di maggiore stress.

2.4.2 - Dimensione eseguibile

Sempre secondo le richieste, un ulteriore parametro analizzato riguarda la dimensione dell'eseguibile del server.

L'eseguibile è stato generato utilizzando il comando:

```
cargo build --release
```

che abilita le ottimizzazioni del compilatore, producendo una versione finale del programma più efficiente in termini di prestazioni e dimensioni.

Nonostante l'utilizzo di diverse librerie esterne, tra cui Axum, Tokio, SQLx e altre dipendenze necessarie, la dimensione dell'eseguibile risulta contenuta.

Questo risultato evidenzia l'efficacia della toolchain di Rust, che attraverso tecniche di ottimizzazione e linking produce binari relativamente compatti, pur mantenendo alte prestazioni e sicurezza.

La dimensione finale dell'eseguibile può variare in base al sistema operativo e all'architettura utilizzata per la compilazione, ma rimane comunque adeguata per il tipo di applicazione sviluppata.

3 - Guida Utente

L'applicazione è composta da due parti principali: backend (server Rust) e client.
Per poter utilizzare correttamente il sistema è necessario seguire i passaggi riportati di seguito.

3.1 - Avvio dell'applicazione

3.1.1 - Preparazione del database

Prima di avviare il server è necessario inizializzare il database eseguendo le migrazioni.

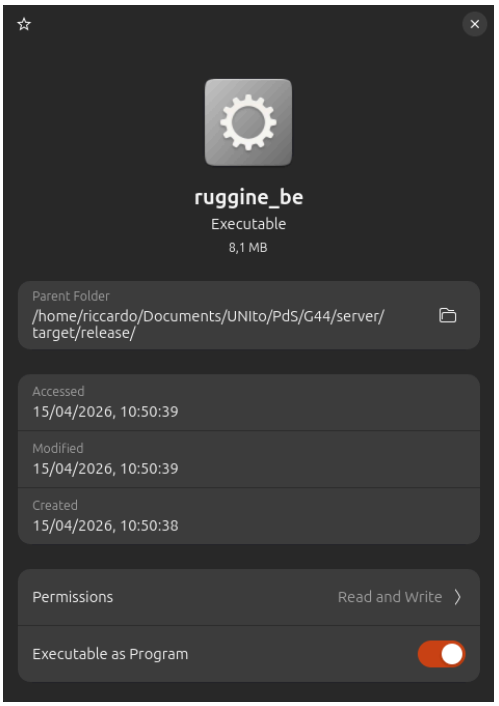
Dalla directory principale del progetto backend, eseguire il comando:
```
sqlx migrate run --database-url sqlite://data/ruggine.db
```

Questo comando crea automaticamente tutte le tabelle necessarie al funzionamento dell'applicazione.

Dopo l'esecuzione delle migrazioni, il database contiene già alcuni utenti di esempio, utili per testare rapidamente l'applicazione senza dover creare nuovi account.

Gli utenti disponibili sono:

- Steve Jobs → username: stewy
- Alan Turing → username: turro
- Bill Gates → username: billy
- Edsger Dijkstra → username: eddy



Tutti gli utenti condividono la stessa password: “password”

Questi account permettono di accedere immediatamente all’applicazione e simulare conversazioni tra utenti diversi, aprendo più sessioni del browser.

3.1.2 - Creazione password (opzionale)

Per inserire manualmente utenti nel database, è possibile generare una password cifrata utilizzando il comando:

```
cargo run --bin mkhash -- "password"
```

Il valore restituito può essere salvato nel database al posto della password in chiaro.

3.1.3 - Avvio del server

Per avviare il backend, posizionarsi nella directory principale del progetto e lanciare:

```
cargo run
```

Il server verrà avviato in locale e resterà in ascolto per le richieste del client.

3.1.4 - Avvio del client

Spostarsi nella directory del client e avviare l’applicazione con:

```
npm run dev
```

Il client sarà accessibile tramite browser all’indirizzo locale indicato dal terminale.

3.1.5 - Utilizzo dell’applicazione

Una volta avviati server e client, è possibile utilizzare la chat tramite browser.

Per simulare l’interazione tra più utenti:

- aprire più finestre del browser (anche in modalità anonima)
- effettuare il login con utenti diversi

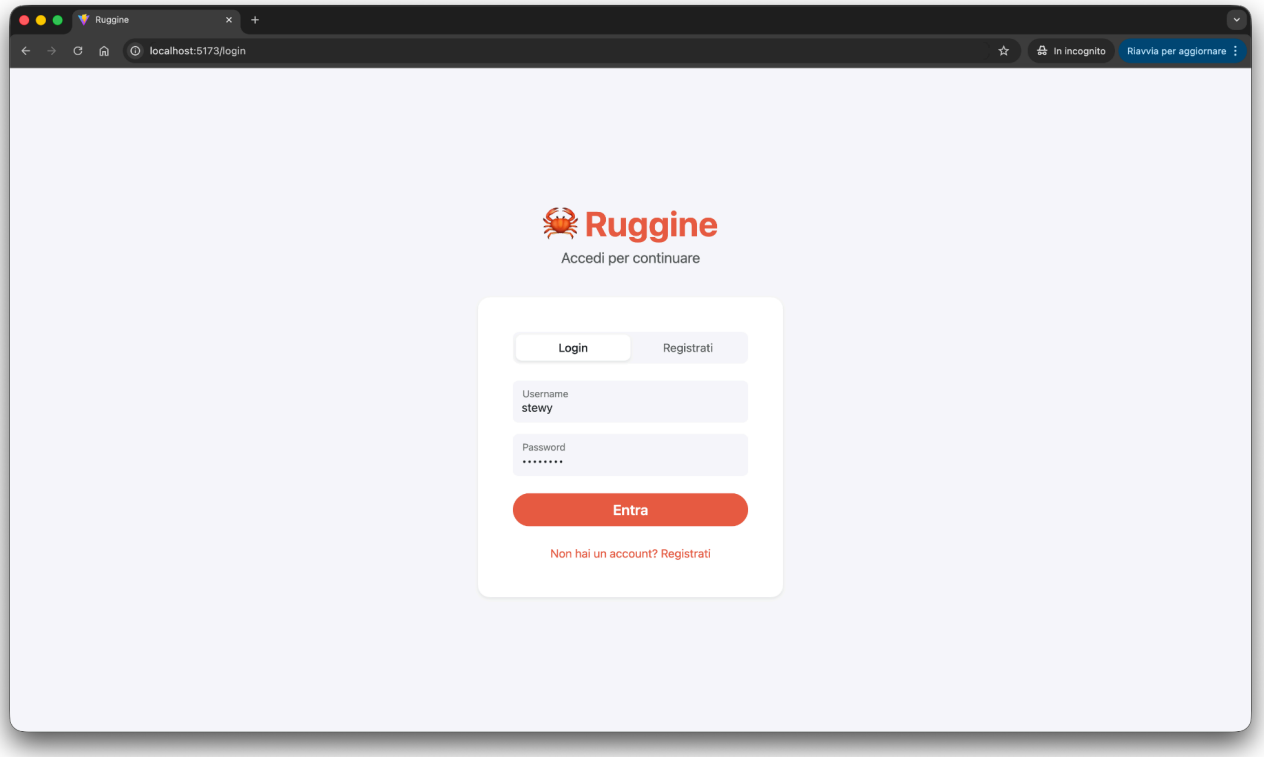
In questo modo è possibile testare correttamente lo scambio di messaggi e le funzionalità di notifica.

Note

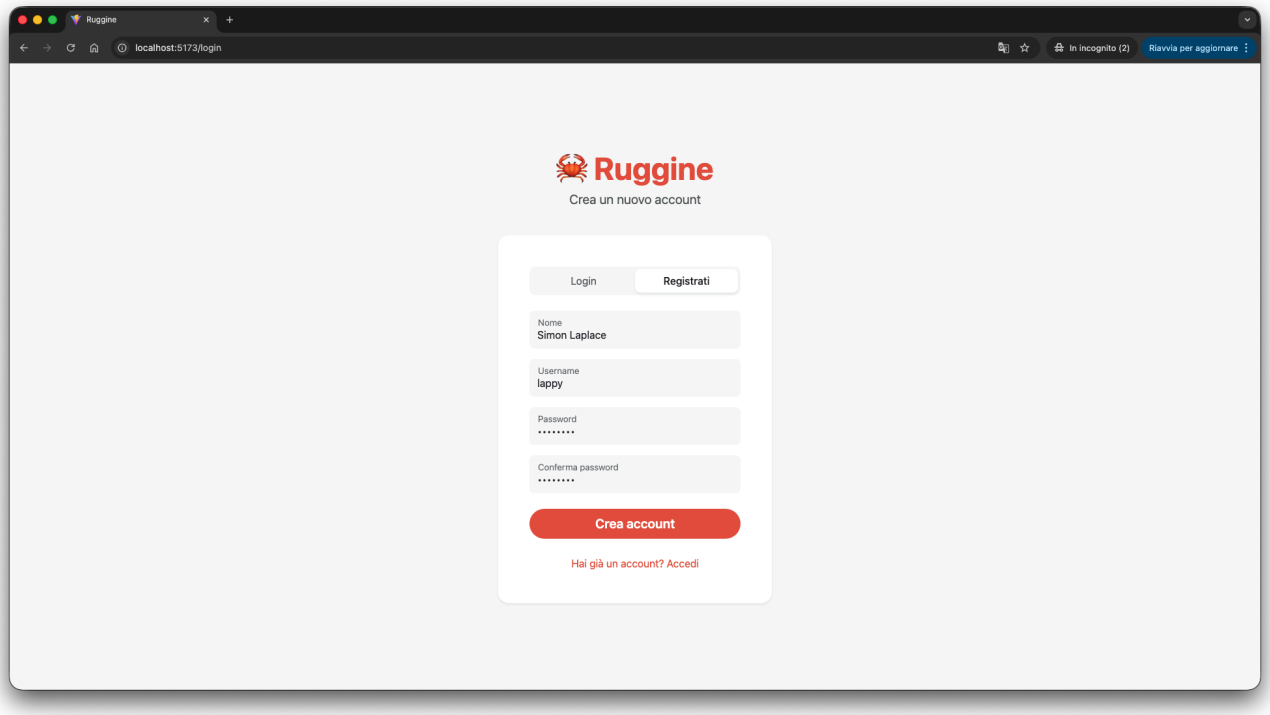
- L’applicazione è pensata per essere eseguita in ambiente locale (localhost).
- La comunicazione avviene tramite protocollo HTTP, sufficiente per il contesto di sviluppo.

3.2 - Descrizione dell’applicazione

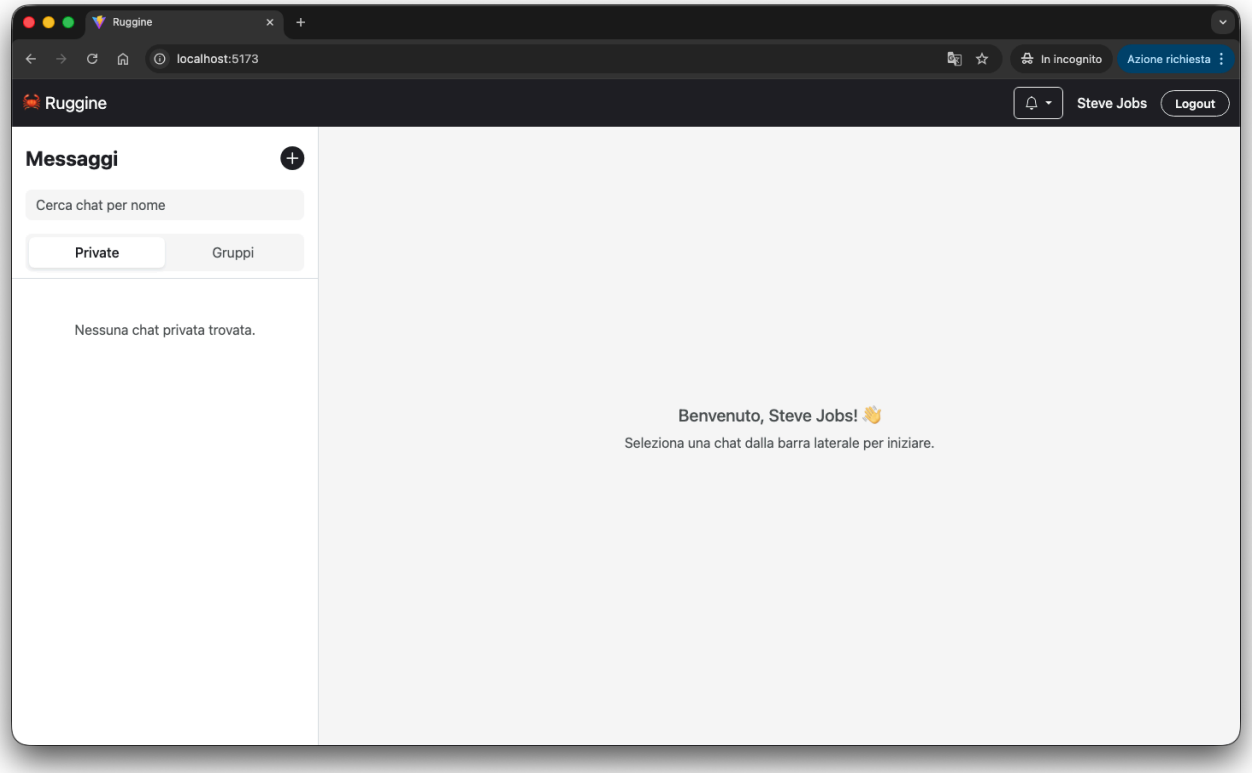
3.2.1 - Login



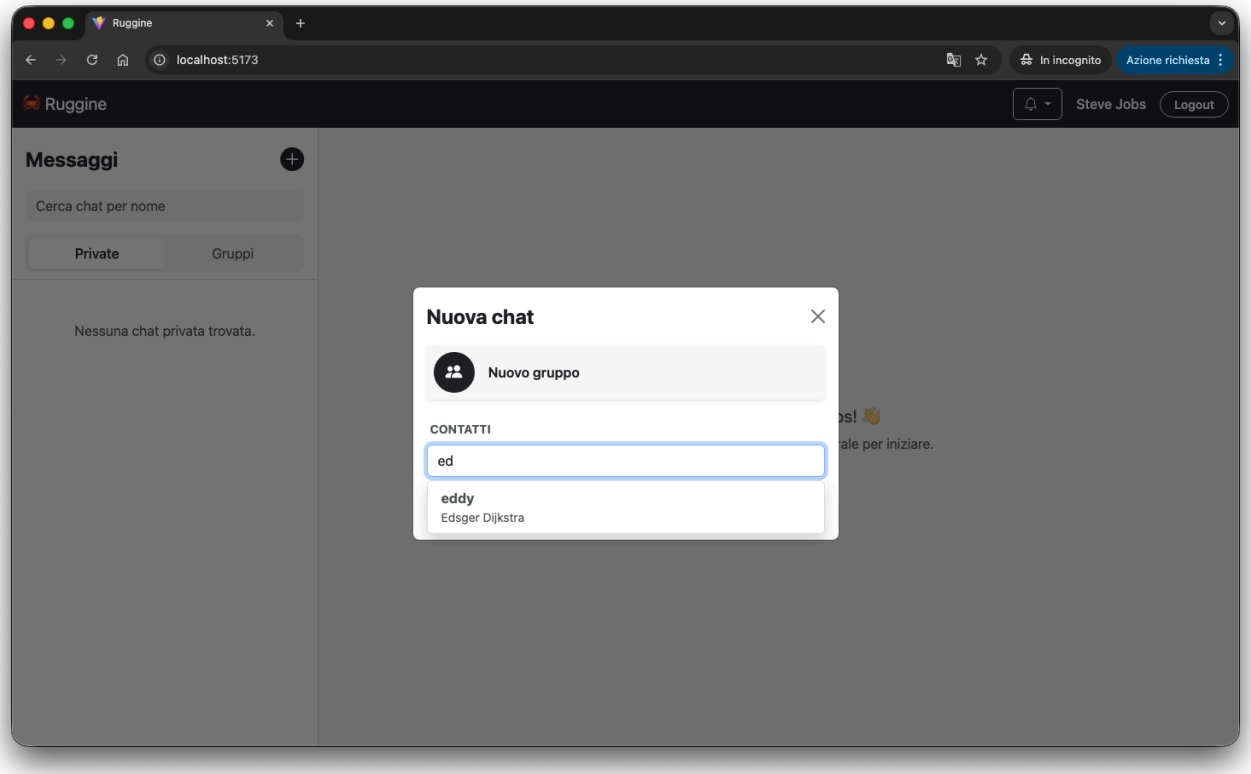
3.2.3 - Registrazione

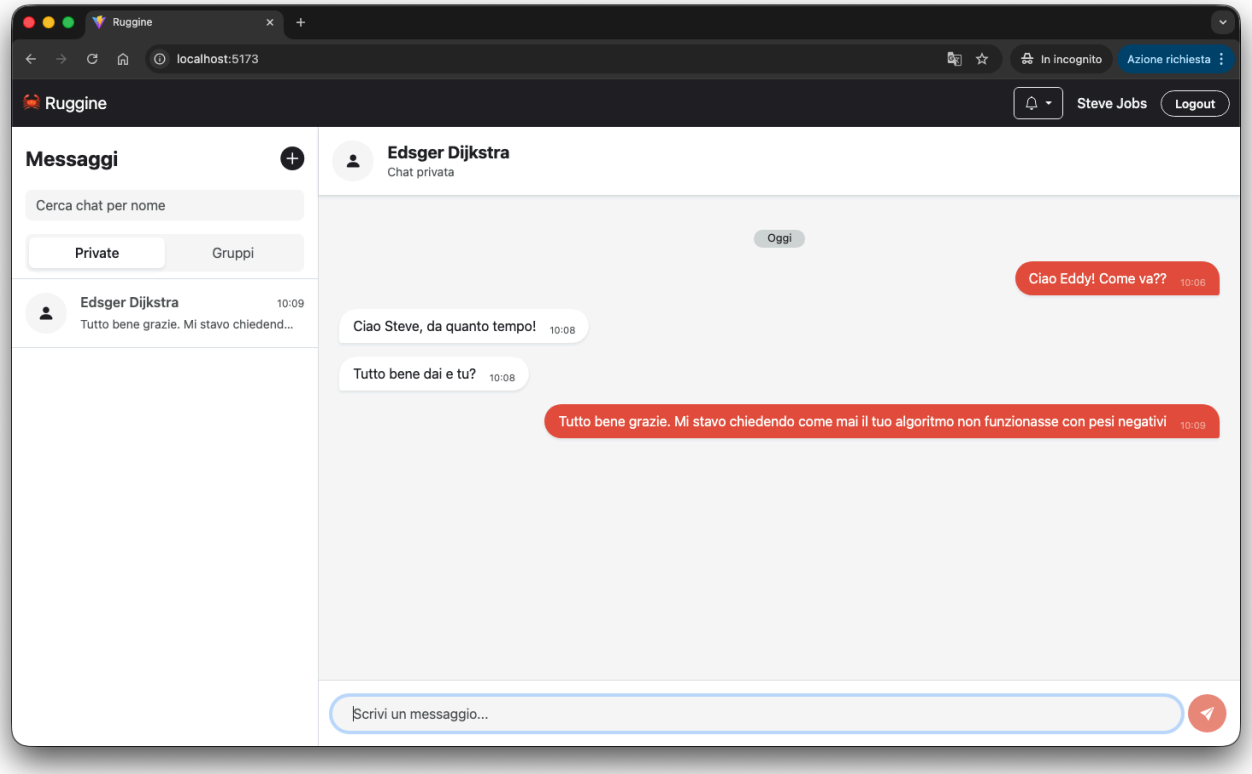


3.2.2 - Landing Page

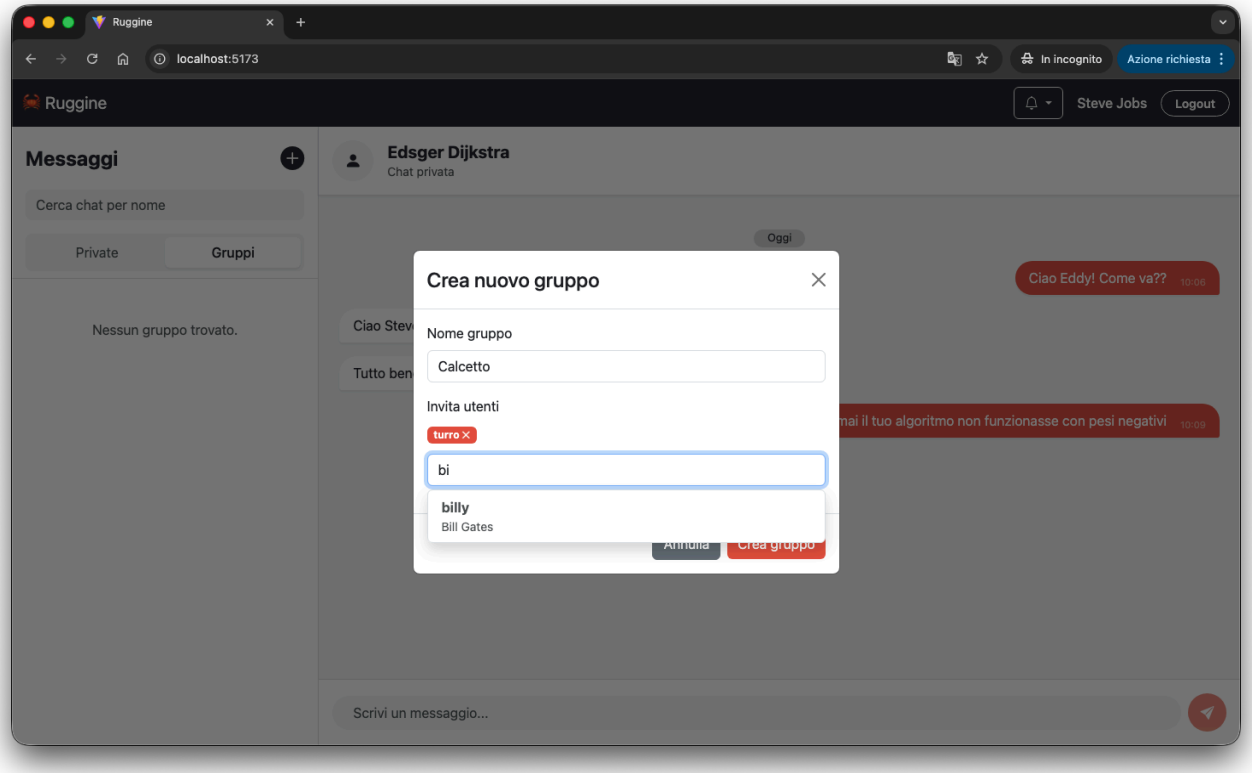


3.2.3 - Chat Privata

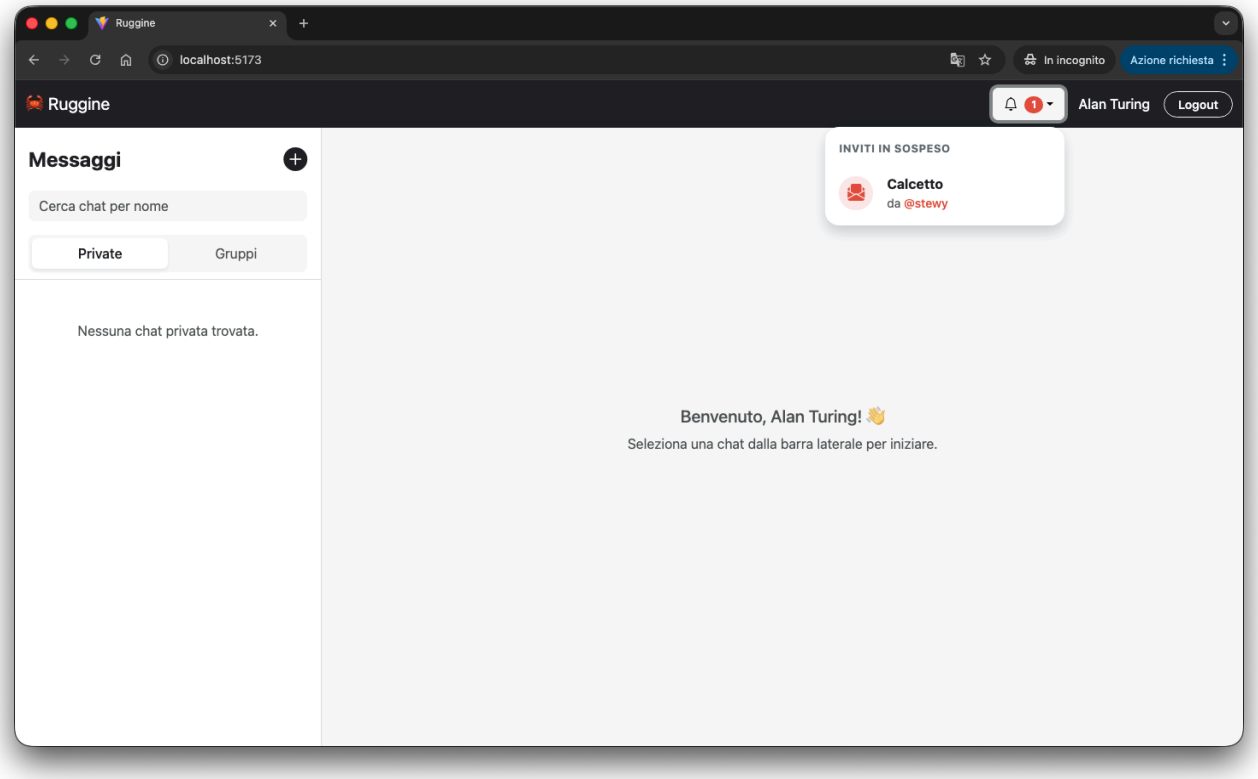


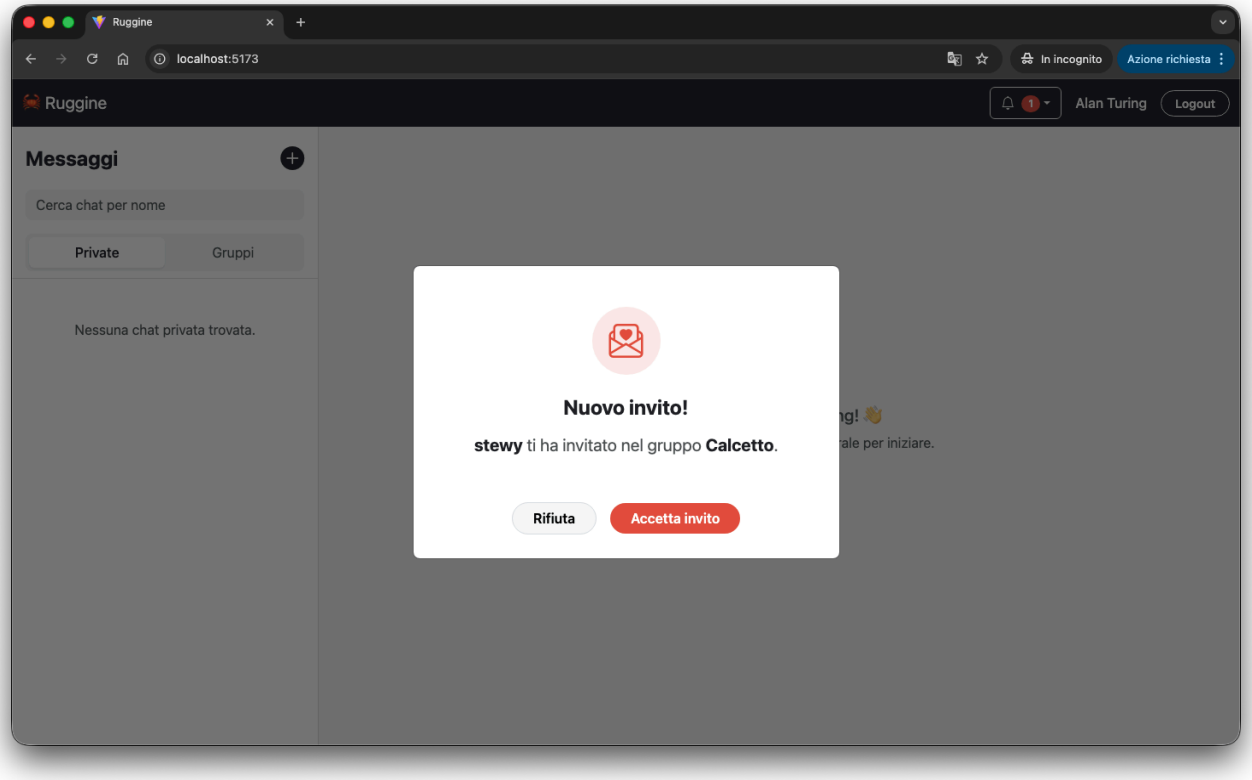


3.2.4 - Creazione Gruppo

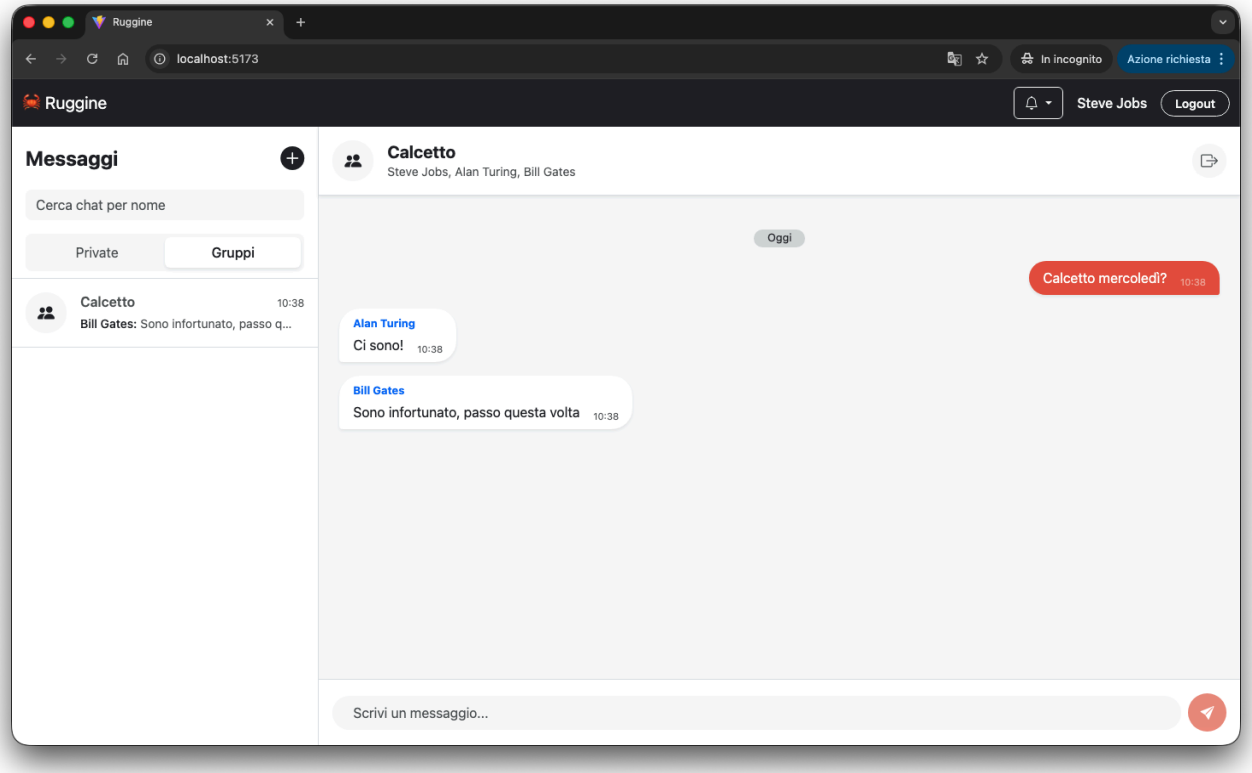
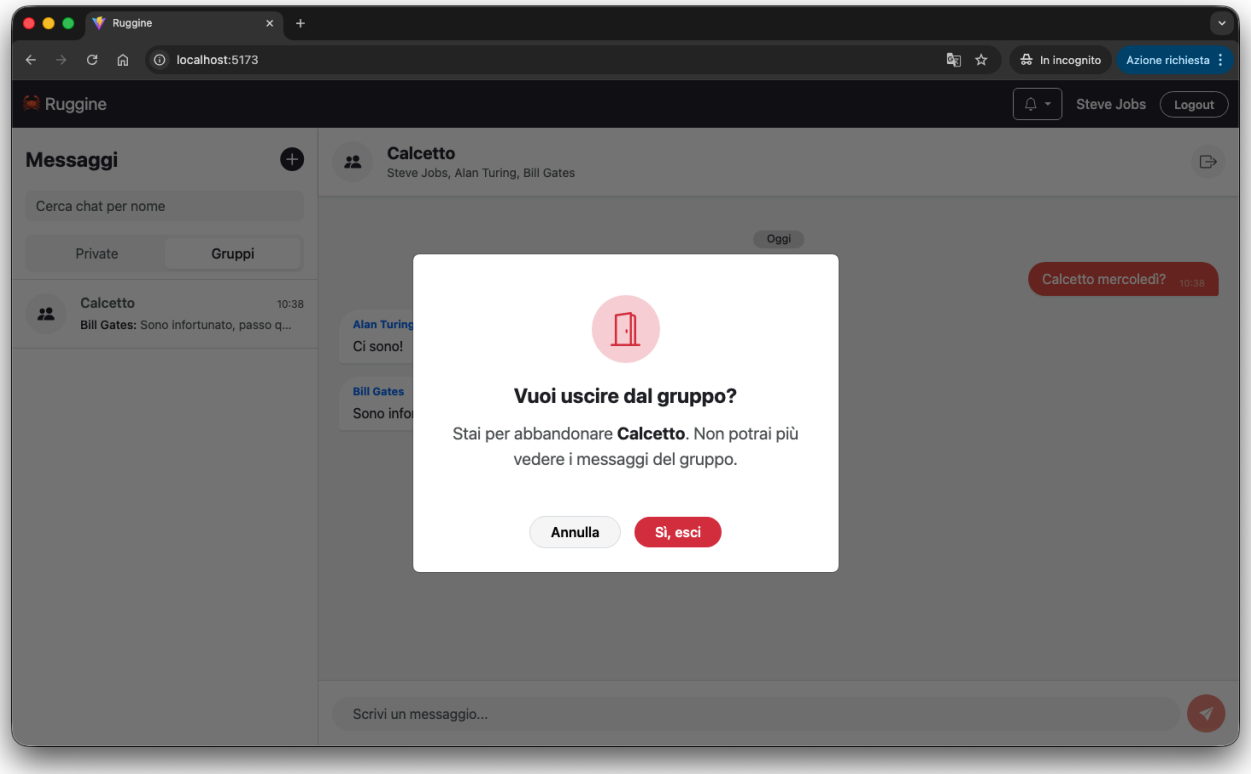


3.2.5 - Lista Inviti e Accettazione Invito





3.2.6 - Chat Di Gruppo



3.2.7 - Abbandono di un Gruppo